IE3142 · DEVOPS SECURITY

Secure Design Principles and Patterns

Lecture 03 · DevSecOps, Secure Desing & Threat Modeling

LECTURE 03

Learning Outcomes

- Explain the importance of secure software development practices.
- Apply secure coding techniques and best practices.
- Describe secure design principles and common security design patterns.
- Explain threat modeling methodologies.
- Perform basic risk assessment during software development.

What Is Secure Design?

Secure design is the process of incorporating security considerations into the software architecture and system design before implementation begins.

It focuses on identifying and eliminating potential security weaknesses early in the Software Development Life Cycle (SDLC), ensuring that security is built into the system rather than added later.

Why Is Secure Design Important?

- Prevents security vulnerabilities before coding begins.
- Reduces the cost and effort required to fix security issues.
- Improves the overall reliability and resilience of software systems.
- Supports the DevSecOps principle of Shift Left Security.
- Provides a strong foundation for secure coding and testing.

What Secure Design Considers

Secure design is the process of incorporating security into the architecture and structure of a system before implementation.

- **Trust boundaries**
- **Data flows**
- **Access control**
- **Component interaction**
- **Failure behaviour**
- **Attack surface**
- **Security assumptions**

Secure Design vs. Secure Coding

Secure Design	Secure Coding
Focuses on the overall architecture and system structure.	Focuses on writing secure source code.
Performed during the design phase of the SDLC.	Performed during the implementation phase.
Identifies security weaknesses before development starts.	Prevents vulnerabilities while writing code.
Example: Designing Role-Based Access Control (RBAC).	Example: Using parameterized SQL queries to prevent SQL Injection.

EXAMPLE · ONLINE BANKING SYSTEM

Design and Coding Working Together

Secure Design Decision

- Only authenticated users can access banking services.
- Customers can view only their own accounts.
- All communication uses HTTPS/TLS.
- Sensitive data is encrypted before storage.
- MFA is required for high-value transactions.

Secure Coding Implementation

- Use parameterized SQL queries.
- Hash passwords with bcrypt or Argon2.
- Validate all user inputs.
- Escape output to prevent XSS.

Secure Design Principles

1. Defence in Depth
2. Fail securely
3. Least privilege
4. Fail-safe defaults
5. Secure defaults
6. Minimize attack surface
7. Separation of duties
8. Complete Mediation
9. Economy of Mechanism
10. Open Design
11. Least Common Mechanism
12. Psychological Acceptability

SECURE DESIGN PRINCIPLE

Secure Design Principle	Description
Defence in Depth	Implement multiple layers of security controls so that if one control fails, other controls continue to protect the system.
Fail Securely	Ensure the system remains in a secure state when an error or failure occurs, preventing unauthorized access or information leakage.
Least Privilege	Grant users, applications, and processes only the minimum permissions required to perform their tasks.
Fail-Safe Defaults	Deny access by default and grant permissions only when they are explicitly authorized.
Secure Defaults	Configure systems with the most secure settings by default, requiring users to intentionally enable less secure options if necessary.
Minimize Attack Surface	Reduce the number of entry points, services, features, and interfaces that attackers can exploit.

SECURE DESIGN PRINCIPLE

Secure Design Principle	Description
Separation of Duties	Divide critical tasks and responsibilities among multiple individuals or systems to reduce the risk of fraud, misuse, or errors.
Complete Mediation	Verify authorization every time a user or process attempts to access a protected resource, rather than relying on previous checks.
Economy of Mechanism	Keep the system design as simple as possible to reduce complexity, implementation errors, and security vulnerabilities.
Open Design	Do not rely on secrecy of the system design for security; instead, security should depend on well-protected secrets such as cryptographic keys.
Least Common Mechanism	Minimize shared resources and mechanisms between users or components to reduce unintended information sharing and attack opportunities.
Psychological Acceptability	Design security controls that are easy to understand and use, encouraging users to follow secure practices rather than bypass them.

Threat Modeling

What Is Threat Modeling?

Threat modeling is a structured process used to identify, analyze and address potential threats before they become exploitable vulnerabilities.

- What are we building?
- What can go wrong?
- What will we do about it?
- Did we address the identified threats properly?

Reference: https://www/owasp.org/index.php/Application_Threat_Modeling

Why Perform Threat Modeling?

Threat modeling helps teams:

- Identify threats early
- Understand assets and trust boundaries
- Prioritize security requirements
- Select appropriate controls
- Reduce expensive redesign
- Improve communication between teams
- Support Shift Left Security
- Document security assumptions

Key Threat Modeling Concepts

Asset

Something valuable that must be protected.

Threat

A potential cause of harm.

Vulnerability

A weakness that can be exploited.

Threat Actor

A person, group or system that may cause harm.

Attack Vector

The path used to exploit a vulnerability.

Mitigation

A control that reduces likelihood or impact.

Threat Modeling Approaches

Methodologies are grouped by the approach they take:

Software-Centric. - Focused on software-based attacks.

Security-Centric. - Focused on known risks and vulnerabilities.

Asset-Centric. - Focused on targets of value.

Threat Modeling Methodologies

Common methodologies include:

STRIDE

Categorizes technical threats.

PASTA

A risk-focused, attack-oriented methodology.

Attack Trees

Represent different paths attackers may use to reach a goal.

Misuse / Abuse Cases

Describe how a system could be intentionally misused.

LINDDUN

Focuses primarily on privacy threats.

DREAD

Estimates the severity of identified threats.

STRIDE Threat Modeling

STRIDE is a threat modeling methodology developed by Microsoft to help identify different categories of security threats in software systems.

Letter	Threat	Security Property Affected
S	Spoofing	Authentication
T	Tampering	Integrity
R	Repudiation	Accountability
I	Information Disclosure	Confidentiality
D	Denial of Service	Availability
E	Elevation of Privilege	Authorization

1. Spoofing & 2. Tampering

S Spoofing

An attacker pretends to be another user or system.

EXAMPLES

- Stolen username and password
- Fake login page
- Forged authentication token

POSSIBLE CONTROLS

- Multi-Factor Authentication (MFA)
- Strong password policies
- Digital certificates

T Tampering

Unauthorized modification of data or software.

EXAMPLES

- Changing grades in a student system
- Modifying transaction amounts
- Editing application files

POSSIBLE CONTROLS

- Hashing
- Digital signatures
- Access control

STRIDE

3. Repudiation & 4. Information Disclosure

R Repudiation

A user denies performing an action because there is no evidence proving otherwise.

EXAMPLES

- A student denies submitting an assignment.
- A lecturer denies changing exam marks.

POSSIBLE CONTROLS

- Audit logs
- Digital signatures
- Time stamps

I Info. Disclosure

Sensitive information becomes accessible to unauthorized users.

EXAMPLES

- Exposed passwords
- Database leaks
- API keys in GitHub repositories

POSSIBLE CONTROLS

- Encryption
- Access control
- Data masking

5. Denial of Service & 6. Elevation of Privilege

D Denial of Service

Attackers prevent legitimate users from accessing services.

EXAMPLES

- Flooding a web server with requests
- Resource exhaustion
- Login brute-force attacks

POSSIBLE CONTROLS

- Rate limiting
- Firewalls
- Load balancing

E Elevation of Privilege

An attacker gains permissions beyond those originally assigned.

EXAMPLES

- Student becomes administrator
- Normal user accesses admin functions

POSSIBLE CONTROLS

- Least Privilege
- RBAC
- Server-side authorization

STRIDE EXAMPLE

Online Banking System

Asset	Threat (STRIDE)	Security Control
Customer Account	Spoofing	MFA
Account Balance	Tampering	Digital Signature
Transaction History	Repudiation	Audit Logs
Personal Information	Information Disclosure	Encryption
Banking Server	Denial of Service	Rate Limiting
Admin Portal	Elevation of Privilege	RBAC

DREAD Risk Assessment

DREAD is a risk assessment model developed by Microsoft that helps estimate the severity of identified threats.

Unlike STRIDE, which identifies types of threats, DREAD helps determine how serious those threats are.

DREAD Evaluates Each Threat Using Five Factors

Letter	Meaning
D	Damage Potential
R	Reproducibility
E	Exploitability
A	Affected Users
D	Discoverability

DREAD Components — Damage & Reproducibility

Damage Potential

How much damage can the attack cause?

- Data loss
- Financial loss
- Reputation damage

Reproducibility

Can attackers repeat the attack easily?

- A vulnerability that works every time receives a high score.

DREAD Components — Exploitability, Users & Discoverability

Exploitability

How difficult is it to perform the attack?

- Does it require advanced knowledge?
- Are special tools required?

Affected Users

How many users could be impacted?

- One user
- One department
- Entire organization

Discoverability

How easy is it to discover the vulnerability?

- Hidden
- Difficult
- Publicly visible

DREAD EXAMPLE

SQL Injection — Risk Score

Category	Score
Damage	9
Reproducibility	10
Exploitability	8
Affected Users	9
Discoverability	7

AVERAGE SCORE

$(9+10+8+9+7) \div 5 = 8.6$

RISK LEVEL

HIGH

While DREAD is useful for learning risk prioritization, many organizations now use other frameworks such as CVSS or organisation-specific risk models because DREAD scoring can be subjective.

COMPARISON

STRIDE vs. DREAD

STRIDE	DREAD
Identifies threats	Measures risk
Used during threat modeling	Used after threats are identified
Focuses on attack categories	Focuses on threat severity
Answers “What can go wrong?”	Answers “How serious is it?”

OWASP and DevSecOps

The Open Worldwide Application Security Project (OWASP) provides open standards, tools, and guidance that help organizations integrate security throughout the DevSecOps lifecycle.

- Shift Left Security
- Secure Software Development Lifecycle (SSDLC)
- Secure coding practices
- Continuous security testing
- Security automation
- Developer security awareness

OWASP resources are widely adopted because they are free, community-driven, and vendor-neutral.

OWASP DevSecOps Guidance

OWASP recommends integrating security into every stage of the CI/CD pipeline.

DevSecOps Stage	OWASP Security Activities
Requirements	Define security requirements
Design	Threat modeling and secure architecture
Development	Secure coding and peer reviews
Build	Static Application Security Testing (SAST)
Test	Dynamic Application Security Testing (DAST), API testing
Release	Security approval and policy checks
Deploy	Secure configuration and secrets management
Operate	Continuous monitoring, vulnerability management, incident response

OWASP

OWASP DevSecOps Maturity Model (DSOMM)

The DSOMM helps organizations evaluate and improve how well security is integrated into their DevSecOps practices. It assesses maturity across:

Build and Deployment	**Implementation (Secure Coding)**	**Information Gathering**
Test and Verification	**Operations**	**Culture and Training**

MATURITY LEVELS

- Level 1: Basic, manual security activities.
- Level 2: Repeatable security processes with partial automation.
- Level 3: Security integrated across the CI/CD pipeline.
- Level 4: Continuous measurement, optimisation, and mature DevSecOps culture.

BENEFITS

- Identifies strengths and weaknesses.
- Guides continuous improvement.
- Encourages security automation.
- Aligns security with software delivery goals.

COMPARISON

STRIDE vs. DREAD vs. OWASP

Framework	Primary Purpose
STRIDE	Identify different categories of security threats.
DREAD	Assess and prioritise the severity of identified threats.
OWASP DevSecOps	Provide best practices, tools, and processes to integrate security throughout the software development lifecycle.

SYNTHESIS

How They Work Together

1. System Design
2. Threat Modeling
3. Identify Threats (STRIDE)
4. Assess Risk (DREAD)
5. Apply OWASP DevSecOps Practices
6. Secure Design
7. Secure Coding
8. SAST / DAST / Code Review
9. Secure Deployment
10. Continuous Monitoring

Q & A

Questions and open discussion.